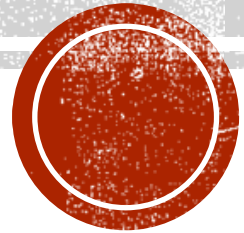
TEMPLATES

Data Structures Fall 2023



FUNCTIONS

- **Main reason for using functions: make pieces of code reusable by encapsulating them within a function.**
- **Example: Interchange the values of two int variables x and y.**
- Instead of inline code:

```
int temp = x;
x = y;
y = temp;
```

write a function:

```
void Swap(int & first, int & second)
{
  int temp = first;
  first = second;
  second = temp;
}
```

GENERAL SOLUTION

- This **function** gives a **general solution** to the **interchange problem for ints** (to exchange the values of any two **integer** variables):

```
Swap(x, y);

    ...

Swap(w, z);

    ...

Swap(a, b);
```

- **NOTE:** ***Inline code would have to be rewritten for each pair of integer variables to swap.***

BUT NOT FOR DOUBLES…

- **To interchange the values of two double variables: *Cannot use the preceding function*; it swaps ints not doubles.**
- **However, *overloading* allows us to define multiple versions of the same function:**

```
/* Function to swap two double variables */
void Swap(double & first, double & second)
{
   double temp = first;
   first = second;
   second = temp;
}
```

- **The two different Swap functions are distinguished by the compiler according to each function's *signature* (name, number, type, and order of parameters).**

Signature

AND FOR STRINGS...

- To **interchange** the **values** of **two string variables**:
- **Again, overload function Swap():**

```
/* Function to swap two string variables*/
void Swap(string & first, string & second)
{
   string temp = first;
   first = second;
   second = temp;
}
```

WHAT ABOUT USER DEFINED TYPES?

- And so on … for other types of variables.
- We would have to overload Swap() for each user-defined type:

```
/* Function to swap two Time variables */
void Swap(Time & first, Time & second)
{
   Time temp = first;
   first = second;
   second = temp;
}
```

OBSERVATIONS:

- The ***logic*** in **each function** is **exactly the same.**
- The **only difference** is in the ***type of the values*** being exchanged.
- If we could **pass the type as an argument**, **we could write a general solution** that could be used to **exchange the values of any two variables**.

GENERIC PROGRAMMING

- Programs containing generic abstractions
- Generic program abstraction (function, class) can be parameterized with a type
  - Abstractions can work with many different types of data
- Algorithms are written in terms of types to-be-specified-later
  - Instantiated when needed for specific types provided as parameters
- Advantages
  - Reusability
  - Writability
  - Maintainability

TEMPLATES

- In C++ generic programming is done using templates

- Two types of templates
  - Function Templates
  - Class Templates

```
template <class Type>
function or class declaration;
```

```
template <typename Type>
function or class declaration;
```

- Compiler generates different type-specific copies from a single template

TEMPLATE MECHANISM

Declare a ***type parameter*** ***(type placeholder)*** and **use it in the function instead** of a **specific type**. This requires a ***different kind of parameter list*** :

```
template <typename DataType >  // type parameter

void Swap( DataType & first, DataType & second)
{
   DataType   temp  = first;
       first = second;
       second = temp;
}
```

TEMPLATE MECHANISM COMMENTS

- The **word** **template** is a **C++ keyword** specifying that what **follows is a pattern** for a **function** ***not a function definition***.

- **Originally**, the **keyword** **class** was **used instead of** **typename** in a **type-parameter list.**

HOW IS A TEMPLATE USED?

`<typename DataType>` names `DataType` as a type parameter — value will be determined by the ***compiler*** from the ***type of the arguments passed when*** **`Swap()`** ***is called***.

Example:

```
#include "Time.h"
#include "Swap.h"   //ONE function template definition
int main()
{
  int    i1, i2;
  double d1, d2;
  string s1, s2;
  Time   t1, t2;
   ...              // Compiler generates definitions
                    // of Swap() with DataType replaced
  Swap(i1, i2);     //       by int
  Swap(d1, d2);     //       by double
  Swap(s1, s2);     //       by string
  Swap(t1, t2);     //       by Time
}
```

GENERAL FORM OF TEMPLATE

```
template <typename TypeParam>
      FunctionDefinition
```

or

```
      template <class TypeParam>
      FunctionDefinition
```

where:

TypeParam **is a type-parameter (placeholder) naming**

the "generic" type of value(s) on which the function operates

FunctionDefinition **is the definition of the function, using type** ***TypeParam*****.**

```
// tempabs.cpp
// template used for absolute value function
#include <iostream>
using namespace std;
//-----------------------------------------------
template <class T>                  //function template
T abs(T n)
    {
    return (n < 0) ? -n : n;
    }
//-----------------------------------------------
int main()
    {
    int int1 = 5;
    int int2 = -6;
    long lon1 = 70000L;
    long lon2 = -80000L;
    double dub1 = 9.95;
    double dub2 = -10.15;
```

EXAMPLE

EXAMPLE

```
    cout << "\nabs(" << int1 << ")=" << abs(int1);   //abs(int)
    cout << "\nabs(" << int2 << ")=" << abs(int2);   //abs(int)
    cout << "\nabs(" << lon1 << ")=" << abs(lon1);   //abs(long)
    cout << "\nabs(" << lon2 << ")=" << abs(lon2);   //abs(long)
    cout << "\nabs(" << dub1 << ")=" << abs(dub1);   //abs(double)
       cout << "\nabs(" << dub2 << ")=" << abs(dub2);   //abs(double)
       cout << endl;
       return 0;
    }
```

Here's the output of the program:

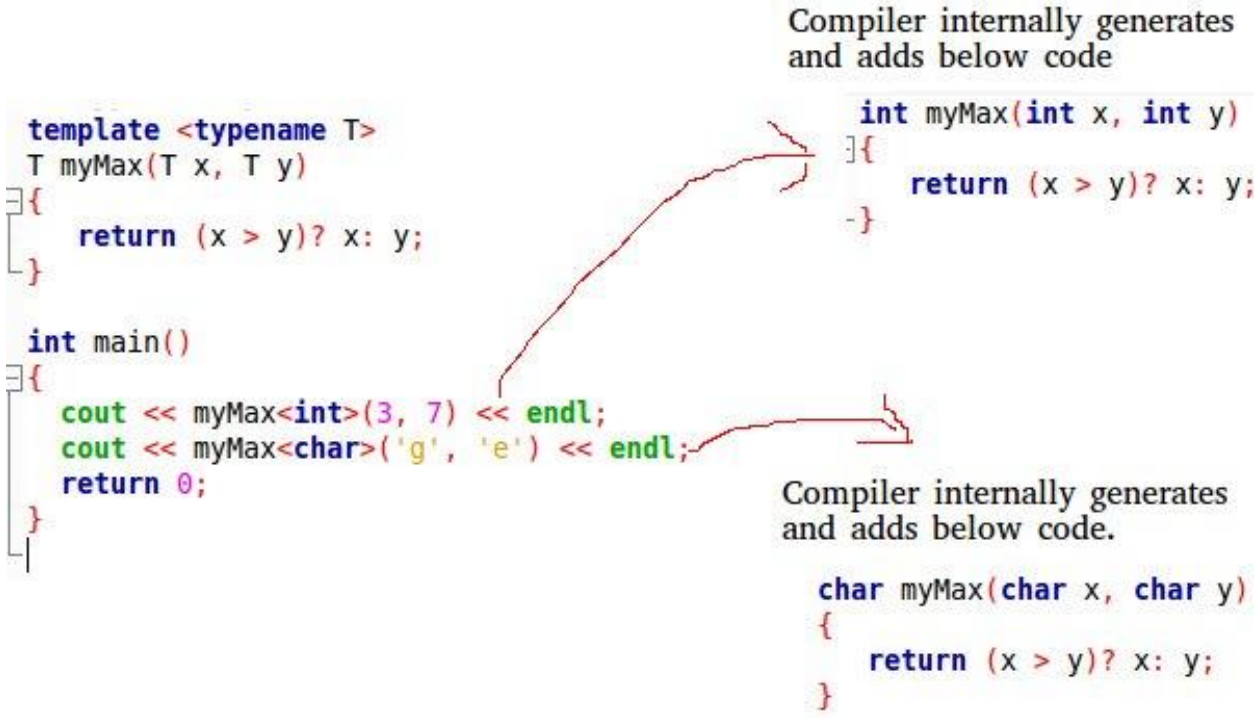
```
abs(5)=5
abs(-6)=6
abs(70000)=70000
abs(-80000)=80000
abs(9.95)=9.95
abs(-10.15)=10.15
```

TEMPLATE INSTANTIATION

- A **function template** is only a **pattern** that **describes how individual functions** can be **built from given actual types**.
- This **process of constructing a function** is called **instantiation**.
- We **instantiated Swap() four times** — with **types int**, **double**, **string**, and **Time**.
- In **each instantiation**, the **type parameter is said to be *bound*** to the **actual type passed**.
- A **template thus serves as a pattern** for the **definition** of an **unlimited number of instances**.

TEMPLATE INSTANTIATION

- The word *template* is a C++ keyword specifying that what follows is a pattern for a function not a function definition.



TEMPLATES WITH MORE THAN ONE PARAMETER

- Like normal parameters, we can pass more than one data types as arguments to templates.

```
template <class T, class U>
class A {
    T x;
    U y;

public:
    A() { cout << "Constructor Called" << endl; }
};

int main()
{
    A<char, char> a;
    A<int, double> b;
    return 0;
}
```

TEMPLATES WITH DEFAULT PARAMETER

- Like normal parameters, we can pass more than one data types as arguments to templates.

```
template <class T = int>
class A {
    T x;

public:
    A() { cout << "Constructor Called" << endl; }
};

int main()
{
    A a; // T is an int by default
    A<double> b; //T is a double

    return 0;
}
```

EXPLICIT TYPE PARAMETERIZATION

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

```
int main() {
    int x;
    x = getInput();              // ??

    double y;
    y = getInput();              // ??
}
```

EXPLICIT TYPE PARAMETERIZATION

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

```
int main() {
    int x;
    x = getInput();              // Error!

    double y;
    y = getInput();              // Error!
}
```

EXPLICIT TYPE PARAMETERIZATION

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

```
int main() {
   int x;
   x = getInput< int >();

   double y;
   y = getInput< double >();
}
```

USER-DEFINED SPECIALIZATION

- A template may not handle all the types successfully
- Explicit specializations might be needed for specific type(s)

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

```
int main( ) {
    isEqual( 5, 6 );           // OK
    isEqual( 7.5, 7.5 );       // OK
    isEqual( “abc”, “xyz” );   // Logical Error!
    return 0;
}
```

USER-DEFINED SPECIALIZATION

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

General Template

```
template< >
bool isEqual< const char* >(const char* x, const char* y ) {
    return ( strcmp( x, y ) == 0 );
}
```

User Specialization

```
int main( ) {
    isEqual( 5, 6 );           // Target: General Template
    isEqual( 7.5, 7.5 );       // Target: General Template
    isEqual( “abc”, “xyz” );   // Target: User Specialization
    return 0;
}
```

CLASS TEMPLATES

- The template concept can be extended to classes
- Write template before class definition
- If the member functions are out-of-line the expression `template<class Type>` must precede not only the class definition, but each out-of-line function as well

CLASS TEMPLATE

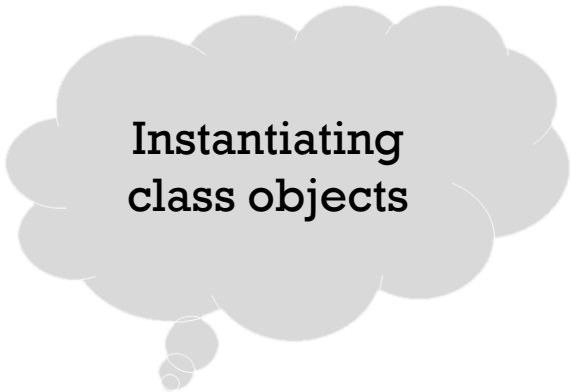
```
template <typename T>
class Array {
private:
    T* ptr;
    int size;

public:
    Array(T arr[], int s);
    void print();
};

template <typename T> Array<T>::Array(T arr[], int s)
{
    ptr = new T[s];
    size = s;
    for (int i = 0; i < size; i++)
        ptr[i] = arr[i];
}

template <typename T> void Array<T>::print()
{
    for (int i = 0; i < size; i++)
        cout << " " << *(ptr + i);
    cout << endl;
}
```

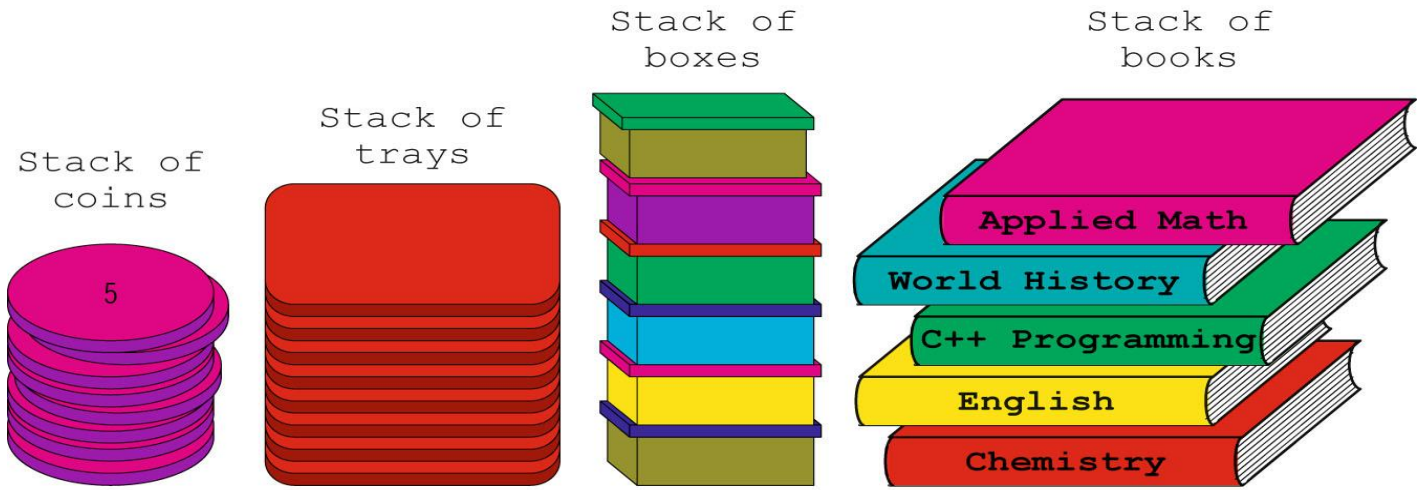
```
int main()
{
    int arr[5] = { 1, 2, 3, 4, 5 };
    Array<int> a(arr, 5);
    a.print();
    return 0;
}
```



EXAMPLES - STACK

- **Stack: a LIFO (last in, first out) data structure**
- **insertions and deletions take place at one end called Top**

- **Examples:**



INTEGER STACK

```
class Stack
   {
   private:
      int st[MAX];          //array of ints
      int top;              //index number of top of stack
   public:
      Stack();              //constructor
      void push(int var);   //takes int as argument
      int pop();            //returns int value
   };
```

LONG STACK

```
class LongStack
   {
   private:
      long st[MAX];          //array of longs
      int top;               //index number of top of stack
   public:
      LongStack();           //constructor
      void push(long var);   //takes long as argument
      long pop();            //returns long value
   };
```

```
#include <iostream.h>
using namespace std;
const int MAX = 100;                    //size of array
////////////////////////////////////////////////////////////
template <class Type>
class Stack
   {
   private:
      Type st[MAX];                     //stack: array of any type
      int top;                          //number of top of stack
   public:
      Stack()                           //constructor
         { top = -1; }
      void push(Type var)               //put number on stack
         { st[++top] = var; }
      Type pop()                        //take number off stack
         { return st[top--]; }
   };
```

```
int main()
   {
   Stack<float> s1;       //s1 is object of class Stack<float>

   s1.push(1111.1F);      //push 3 floats, pop 3 floats
   s1.push(2222.2F);
   s1.push(3333.3F);
   cout << "1: " << s1.pop() << endl;
   cout << "2: " << s1.pop() << endl;
   cout << "3: " << s1.pop() << endl;

   Stack<long> s2;        //s2 is object of class Stack<long>

   s2.push(123123123L);   //push 3 longs, pop 3 longs
   s2.push(234234234L);
   s2.push(345345345L);
   cout << "1: " << s2.pop() << endl;
   cout << "2: " << s2.pop() << endl;
   cout << "3: " << s2.pop() << endl;
   return 0;
   }
```

CLASS TEMPLATE

- If the **member functions** are **defined externally** (**outside of the class specification**), we **need a new syntax**.
- The expression **template<class Type>** must **precede not only the class definition**, but ***each externally defined member function as well***.

```
template<class Type>
void Stack<Type>::push(Type var)
   {
   st[++top] = var;
   }
```

GENERAL FORM OF CLASS TEMPLATE DECLARATION

```
template <typename TypeParam> or
template <class TypeParam>
 class SomeClass
 {
  // ... members of SomeClass ...
 };
```

More than one type parameter may be specified:

```
template <typename TypeParam1,...,
          typename TypeParamn>
class SomeClass
{
  // ... members of SomeClass ...
};
```

Can also be set to a default value

INSTANTIATING CLASS TEMPLATES

- To use a class template in a program/function/library:

  Instantiate it by using a declaration of the form

  ```
  ClassName<Type> object;
  ```

  to **pass `Type` as an argument** to the class template definition.

- Examples:

- ```
  Stack<int>    intSt;
  ```

- ```
  Stack<string> stringSt;
  ```

- Compiler will generate **two distinct definitions** of **`Stack`** — two **instances** — one for **`int`**s and one for **`string`**s.

A LINKED LIST CLASS USING TEMPLATES

```
// templist.cpp
// implements linked list as a template
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////
template<class TYPE>                    //struct link<TYPE>
struct link                             //one element of list
//within this struct definition 'link' means link<TYPE>
   {
   TYPE data;                           //data item
   link* next;                          //pointer to next link
   };
////////////////////////////////////////////////////////////
```

```
template<class TYPE>                        //class linklist<TYPE>
class linklist                              //a list of links
//within this class definition 'linklist' means linklist<TYPE>
   {
   private:
      link<TYPE>* first;                    //pointer to first link
   public:
      linklist()                            //no-argument constructor
         { first = NULL; }                  //no first link

    //note: destructor would be nice; not shown for simplicity
    void additem(TYPE d);                   //add data item (one link)
    void display();                         //display all links
};
```

```
template<class TYPE>
void linklist<TYPE>::additem(TYPE d)  //add data item

   {
   link<TYPE>* newlink = new link<TYPE>;  //make a new link
   newlink->data = d;                     //give it data
   newlink->next = first;                 //it points to next link
   first = newlink;                       //now first points to this
   }
//--------------------------------------------------------------
 template<class TYPE>
 void linklist<TYPE>::display()          //display all links
    {
    link<TYPE>* current = first;         //set ptr to first link
    while( current != NULL )             //quit on last link
       {
       cout << endl << current->data;    //print data
       current = current->next;          //move to next link
       }
    }
```

```
int main()
   {
   linklist<double> ld; //ld is object of class linklist<double>

   ld.additem(151.5);   //add three doubles to list ld
   ld.additem(262.6);
   ld.additem(373.7);
   ld.display();        //display entire list ld
   linklist<char> lch;  //lch is object of class linklist<char>

   lch.additem('a');    //add three chars to list lch
   lch.additem('b');
   lch.additem('c');
   lch.display();       //display entire list lch
   cout << endl;
   return 0;
   }
```

```
template <class myType>
myType GetMax (myType a, myType b) {
 return (a>b?a:b);
}
```

```
// function template
#include <iostream>
using namespace std;

template <class T>
T GetMax (T a, T b) {
  T result;
  result = (a>b)? a : b;
  return (result);
}

int main () {
  int i=5, j=6, k;
  long l=10, m=5, n;
  k=GetMax<int>(i,j);
  n=GetMax<long>(l,m);
  cout << k << endl;
  cout << n << endl;
  return 0;
}
```

```
// class templates
#include <iostream>
using namespace std;

template <class T>
class mypair {
    T a, b;
  public:
    mypair (T first, T second)
      {a=first; b=second;}
    T getmax ();
};

template <class T>
T mypair<T>::getmax ()
{
  T retval;
  retval = a>b? a : b;
  return retval;
}

int main () {
  mypair <int> myobject (100, 75);
  cout << myobject.getmax();
  return 0;
}
```

```
template <class T>
class mypair {
    T values [2];
  public:
    mypair (T first, T second)
    {
      values[0]=first; values[1]=second;
    }
};
```

MULTIPLE TYPE ARGUMENTS

```
template< typename T, typename U >
T my_cast( U u ) {
    return (T)u;
}
```

```
int main() {
    double d = 10.5674;
    int j = my_cast( d );     //Error
    int i = my_cast< int >( d );
    return 0;
}
```

USER-DEFINED TYPES

- Besides primitive types, user-defined types can also be passed as type arguments to templates
- Compiler performs static type checking to diagnose type errors

- Example: Consider String class without overloaded operator "=="

```
class String {
    char* pStr;
    . . .
    // Operator "==" not defined
};
```

USER-DEFINED TYPES

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

```
int main() {
    String s1 = “xyz”, s2 = “xyz”;
    isEqual( s1, s2 );        // Error!
    return 0;
}
```

USER-DEFINED TYPES

```
class String {
    char* pStr;
    . . .
    friend bool operator ==(const String&, const String& );
};

bool operator ==( const String& x, const String& y ) {
    return strcmp(x.pStr, y.pStr) == 0;
}
```

```
int main() {
    String s1 = “xyz”, s2 = “xyz”;
    isEqual( s1, s2 );        // OK
    return 0;
}
```

OVERLOADING VS. TEMPLATES

Function overloading

- Different data types, similar operation
- Example
  - '+' operation is overloaded for different operand types

Function templates

- Different data types, identical operation
- Example
  - A single function template can calculate sum of array of many types

OVERLOADING VS. TEMPLATES – EXAMPLE

```
String operator +( const String& x, const String& y ) {
    String tmp;
    tmp.pStr = new char[strlen(x.pStr) + strlen(y.pStr) + 1 ];
    strcpy( tmp.pStr, x.pStr );
    strcat( tmp.pStr, y.pStr );
    return tmp;
}
```

```
String operator +( const char * str1, const String& y ) {
    String tmp;
    tmp.pStr = new char[ strlen(str1) + strlen(y.pStr) + 1 ];
    strcpy( tmp.pStr, str1 );
    strcat( tmp.pStr, y.pStr );
    return tmp;
}
```

OVERLOADING VS. TEMPLATES – EXAMPLE

```
template< class T >
T sum( T* array, int size ) {
    T sum = 0;

    for (int i = 0; i < size; i++){
        sum = sum + array[i];
    }

    return sum;
}
```

CLASS TEMPLATE

- A class template provides functionality to operate on different types of data
  - Facilitates reuse of classes

- Declaration:

```
template< class T >
class XYZ{ . . .};
```

```
template< typename T >
Class XYZ { . . . };
```

CLASS TEMPLATE – EXAMPLE

- A Vector class template can store data elements of different types
  - Without templates, separate Vector class is needed for each data type

```
template< class T >
class Vector {
private:
    int size;
    T* ptr;
public:
    Vector<T>( int = 10 );
    Vector<T>( const Vector< T >& );
    ~Vector<T>();
    int getSize() const;
    const Vector< T >& operator =(const Vector< T >& );
    T& operator []( int );
};
```

CLASS TEMPLATE – EXAMPLE

```
template< class T >
Vector<T>::Vector( int s ) {
    size = s;
    if ( size != 0 )
        ptr = new T[size];
    else
        ptr = 0;
}
```

CLASS TEMPLATE – EXAMPLE

```
template< class T >
Vector<T>:: Vector(const Vector<T>& copy ) {
    size = copy.getSize();
    if (size != 0) {
        ptr = new T[size];
        for (int i = 0; i < size; i++)
            ptr[i] = copy.ptr[i];
    }
    else ptr = 0;
}
```

CLASS TEMPLATE – EXAMPLE

```
template< class T >
Vector<T>::~Vector<T>() {
    delete [] ptr;
}

template< class T >
int Vector<T>::getSize() const {
    return size;
}
```

CLASS TEMPLATE – EXAMPLE

```
template< class T >
const Vector<T>& Vector<T>::operator =( const Vector<T>& right) {
    if ( this != &right ) {
        delete [] ptr;
        size = right.size;
        if ( size != 0 ) {
            ptr = new T[size];
            for(int i = 0; i < size;i++)
                ptr[i] = right.ptr[i];
        }
        else
            ptr = 0;
    }
    return *this;
}
```

CLASS TEMPLATE – EXAMPLE

```
template< class T >
T& Vector< T >::operator [](int index ) {
    if ( index < 0 || index >= size ) {
        cout << “Error: index out of range\n”;
        exit( 1 );
    }
    return ptr[index];
}
```

CLASS TEMPLATE – USAGE

```
int main() {
    Vector< int > iv1( 2 );
    iv1[0] = 15;
    iv1[1] = 27;
    Vector< int > iv2( iv1 );
    Vector< int > iv3( 2 );
    iv3 = iv1;
    return 0;
}
```

CLASS TEMPLATE SPECIALIZATION

```
int main() {
    Vector< char* > sv1( 2 );
    sv1[0] = "Aamir";
    sv1[1] = "Nasir";
    Vector< char* > sv2( sv1 );
    Vector< char* > sv3( 2 );
    sv3 = sv1;
    return 0;
}
```

- Like function templates, a class template may not handle all the types successfully
- Explicit specializations are provided to handle such types



CLASS TEMPLATE SPECIALIZATION

```
template<>
class Vector< char* > {
private:
    int size;
    char** ptr;
public:
    // Vector< char* >( int = 10 );
    Vector( int = 10 );
    Vector( const Vector< char* >& );
    virtual ~Vector();
    int getSize() const;
    const Vector< char* >& operator =(const Vector< char* >& );
    const char*& operator [](  int );
    void insert( char*, int );
};
```

CLASS TEMPLATE SPECIALIZATION

```
template<>
Vector<char*>::Vector(int s) {
    size = s;
    if ( size != 0 ) {
        ptr = new char*[size];
        for (int i = 0; i < size; i++)
            ptr[i] = 0;
    }
    else
        ptr = 0;
}
```

CLASS TEMPLATE SPECIALIZATION

```
template<>
Vector< char* >::Vector(const Vector<char*>& copy ) {
    size = copy.getSize();
    if ( size == 0 ) {
        ptr = 0;
        return;
    }
    ptr = new char*[size];
    for (int i = 0; i < size; i++)
        if ( copy.ptr[i] != 0 ) {
            ptr[i] = new char[ strlen( copy.ptr[i] ) + 1 ];
            strcpy(ptr[i], copy.ptr[i]);
        }
        else
            ptr[i] = 0;
}
```



CLASS TEMPLATE SPECIALIZATION

```
template<>
Vector<char*>::~Vector() {
    for (int i = 0; i < size; i++)
        delete [] ptr[i];
    delete [] ptr;
}

template<>
int Vector<char*>::getSize() const {
    return size;
}
```

CLASS TEMPLATE SPECIALIZATION

```
template<>
const Vector<char*>& Vector<char*>::operator=(const Vector<char*>& right) {
    if ( this == &right )
        return *this;
    for (int i = 0; i < size; i++)
        delete [] ptr[i];
    delete [] ptr;
    size = right.size;
    if ( size == 0 ) {
        ptr = 0;
        return *this;
    }
    ptr = new char*[size];
    for (int i = 0; i < size; i++)
        if ( right.ptr[i] != 0 ) {
            ptr[i] = new char[strlen( right.ptr[i] ) + 1];
            strcpy( ptr[i], right.ptr[i] );
        }
        else ptr[i] = 0;
}
```

CLASS TEMPLATE SPECIALIZATION

```
template<>
const char*& Vector<char*>::operator []( int index ) {
    if ( index < 0 || index >= size ) {
        cout << "Error: index out of range\n";
        exit( 1 );
    }
    return ptr[index];
}
```

CLASS TEMPLATE SPECIALIZATION

```
template<>
void Vector< char* >::insert(char* str, int i ) {
    delete [] ptr[i];
    if ( str != 0 ) {
        ptr[i] = new char[strlen(str)+ 1];
        strcpy( ptr[i], str );
    }
    else
        ptr[i] = 0;
}
```

CLASS TEMPLATE SPECIALIZATION

```
int main() {
    Vector< char* > sv1( 2 );
    sv1[0] = “Aamir”; // Error
    sv1.insert( "Aamir", 0 );
    sv1.insert( "Nasir", 1 );
    Vector< char* > sv2( sv1 );
    Vector< char* > sv3( 2 );
    sv3 = sv1;
    return 0;
}
```

MEMBER TEMPLATES

- A class or class template can have member functions that are themselves templates.

- Example

```
template <typename T>
class Complex {
    T real, imag;
public:
    // Complex<T>( T r, T im )
    Complex( T r, T im ) : real(r), imag(im) {}
    // Complex<T>(const Complex<T>& c)
    Complex(const Complex<T>& c): real(c.real), imag(c.imag) {}
    . . .
};
```

MEMBER TEMPLATES

```
template <typename T>
class Complex {
    T real, imag;
public:
    // Complex<T>( T r, T im )
    Complex( T r, T im ) : real(r), imag(im) {}
    // Complex<T>(const Complex<T>& c)
    Complex(const Complex<T>& c): real(c.real), imag(c.imag) {}
    . . .
};
```

```
int main() {
    Complex< float > fc( 0, 0 );
    Complex< double > dc = fc; // Error
    return 0;
}
```

MEMBER TEMPLATES – REASON FOR ERROR

```
int main() {
    Complex< float > fc( 0, 0 );
    Complex< double > dc = fc; // Error
    return 0;
}
```

```
class Complex<double> {
    double real, imag;
public:
    Complex( double r, double im ) : real(r), imag(im) {}
    Complex(const Complex<double>& c):real(c.real),imag(c.imag) {}
    . . .
};
```

MEMBER TEMPLATES

```
template <typename T>
class Complex {
    T real, imag;
public:
    Complex( T r, T im ) : real(r), imag(im) {}
    template <typename U>
    Complex(const Complex<U>& c):real( c.real ), imag( c.imag ) {}
    . . .
};
```

MEMBER TEMPLATES

```
int main() {
    Complex< float > fc( 0, 0 );
    Complex< double > dc = fc; // OK
    return 0;
}
```

```
class Complex< double> {
    double real, imag;
public:
    Complex( double r, double im ) : real(r), imag(im) {}
    template <typename U>
    Complex(const Complex<U>& c):real( c.real), imag( c.imag ) {}
    . . .
};
```